Introduction

Report on Airport Navigation System Using Dijkstra's Algorithm

The Airport Navigation System is a user-friendly application designed to assist travelers in efficiently navigating an airport. It employs Dijkstra's algorithm to determine the shortest path between various locations within the airport. The system allows users to add locations and paths, find the shortest route, and visualize the airport layout through an interactive graphical interface.

Literature Review

Navigation systems have significantly evolved with the advancement of technology. Early systems relied on paper maps and basic directional signs, which often led to confusion and inefficiencies. With the rise of algorithms like Dijkstra's, navigation has become more accurate and user-friendly. Dijkstra's algorithm, developed by Edsger Dijkstra in 1956, is widely used in network routing and geographic information systems (GIS). Its efficiency in finding the shortest path in weighted graphs has made it a cornerstone in applications ranging from GPS navigation to robotics.

Recent studies emphasize the importance of integrating user interfaces with sophisticated algorithms to enhance user experience. Effective visualization tools are critical in making complex data comprehensible, especially in large-scale environments like airports.

Implementation Details

The Airport Navigation System is implemented using Python, leveraging the Tkinter library for the graphical user interface (GUI) and Matplotlib with NetworkX for graph visualization. The core functionality is encapsulated in two classes: `DijkstraApp` for the GUI and `Graph` for managing the graph structure and implementing the Dijkstra algorithm.

Key Features

1. Node and Edge Management: Users can add locations (nodes) and paths (edges) with distances, which are stored in an adjacency list.

2. Shortest Path Calculation: The system allows users to find the shortest route from a specified starting location to all other locations using Dijkstra's algorithm.

3. Graph Visualization: The application includes a feature to visually represent the airport layout, enhancing user comprehension of the navigation paths.

System Study

The application operates in a straightforward manner:
1. Users input locations and paths through the GUI.
2. The graph structure is updated dynamically based on user inputs.
3. The shortest path is computed when requested, and results are displayed in a text box.
4. Users can visualize the graph to understand the airport layout better.

The system is designed to be scalable, accommodating an increasing number of nodes and edges as more locations and paths are added. The underlying graph algorithm ensures that the performance remains optimal for a reasonably sized airport layout.

Technical Feasibility

The technical feasibility of the Airport Navigation System is high. The application is developed using widely adopted libraries and programming languages. Python's Tkinter provides a reliable framework for building desktop applications, while NetworkX and Matplotlib are well-suited for graph manipulation and visualization.

Hardware and Software Requirements:

- Hardware: A computer or laptop with standard specifications (minimum 4GB RAM).
- Software: Python (version 3.x), Tkinter, Matplotlib, and NetworkX libraries.

Given the simplicity of the application and the availability of the required technologies, implementing this system is feasible within standard development timelines

.

Code

For Gui :

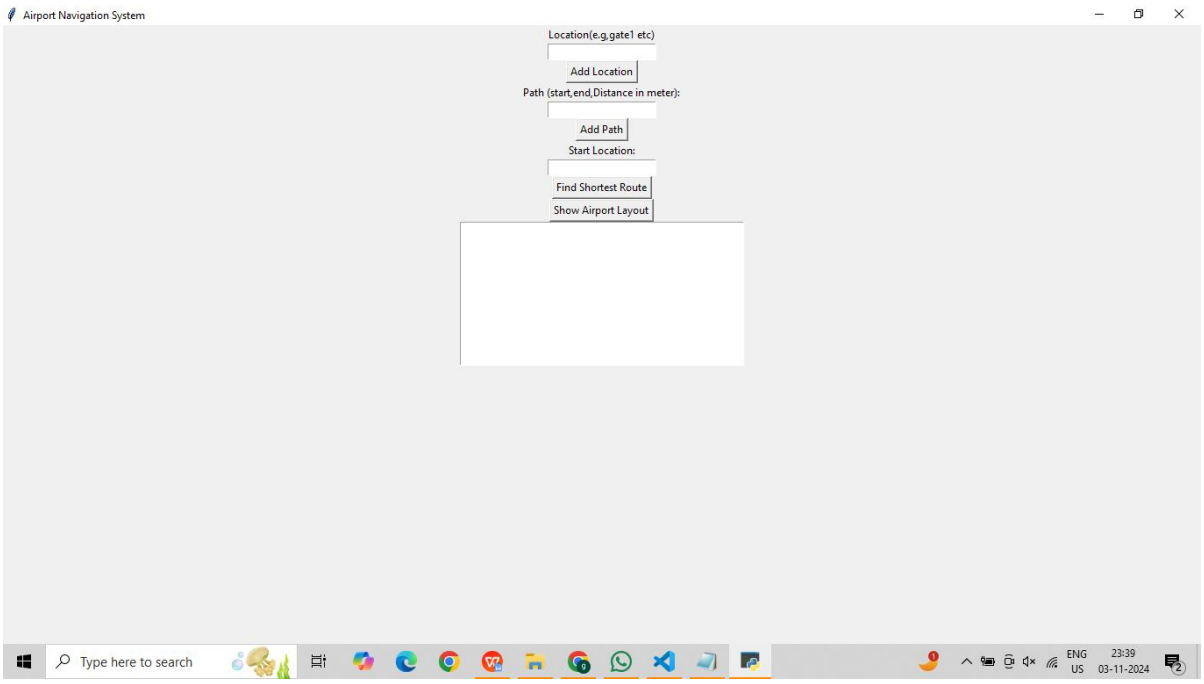
```
import tkinter as tk
from tkinter import messagebox
from dijkstra import Graph
import matplotlib.pyplot as plt
import networkx as nx

class DijkstraApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Airport Navigation System")

        self.graph = Graph()

        self.node_label = tk.Label(root, text="Location(e.g,gate1 etc)")
        self.node_label.pack()
        self.node_entry = tk.Entry(root)
        self.node_entry.pack()
        self.add_node_button = tk.Button(root, text="Add Location", command=self.add_node)
        self.add_node_button.pack()

        self.edge_label = tk.Label(root, text="Path (start,end,Distance in meter):")
        self.edge_label.pack()
        self.edge_entry = tk.Entry(root)
        self.edge_entry.pack()
        self.add_edge_button = tk.Button(root, text="Add Path", command=self.add_edge)
        self.add_edge_button.pack()

        self.start_label = tk.Label(root, text="Start Location:")
        self.start_label.pack()
        self.start_entry = tk.Entry(root)
        self.start_entry.pack()
        self.find_path_button = tk.Button(root, text="Find Shortest Route",
command=self.find_shortest_path)
        self.find_path_button.pack()

        self.plot_graph_button = tk.Button(root, text="Show Airport Layout", command=self.plot_graph)
        self.plot_graph_button.pack()

        self.result_text = tk.Text(root, height=10, width=40)
        self.result_text.pack()

    def add_node(self):
        node = self.node_entry.get().strip()
        if node:
            self.graph.add_node(node)
            messagebox.showinfo("Info", f"Location {node} added.")
```

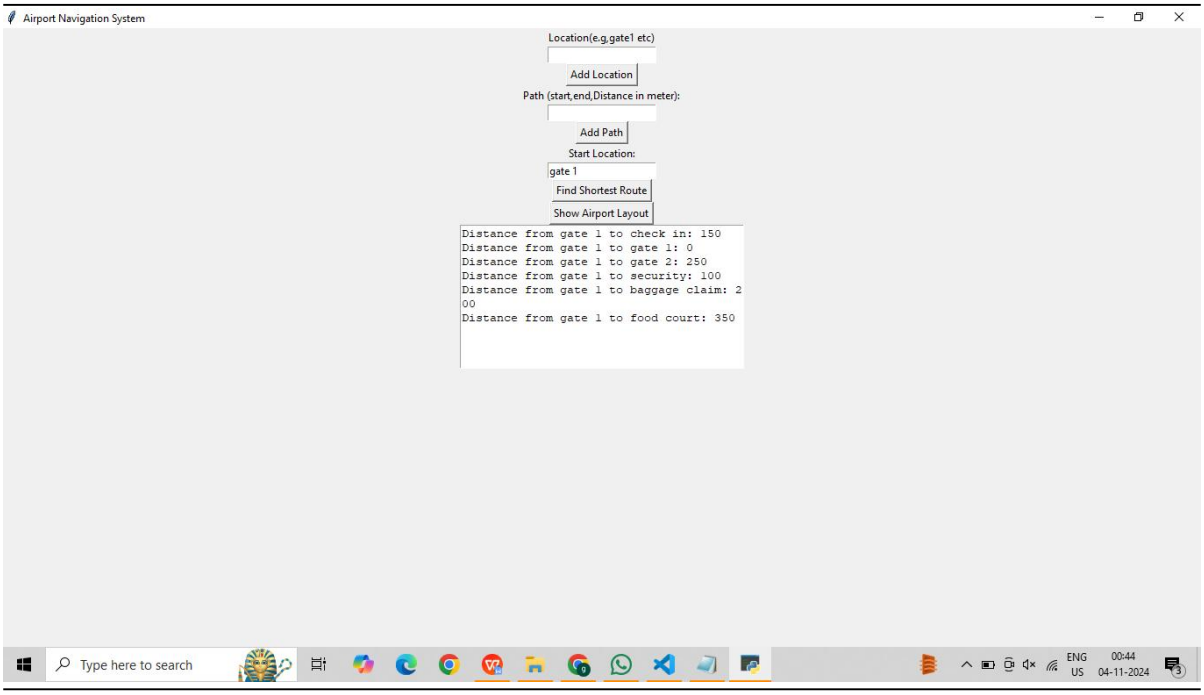
```
        self.node_entry.delete(0, tk.END)

    def add_edge(self):
        edge_data = self.edge_entry.get().strip().split(',')
        if len(edge_data) == 3:
            start, end, weight = edge_data
            try:
                weight = int(weight)
                self.graph.add_edge(start.strip(), end.strip(), weight)
                messagebox.showinfo("Info", f"Edge from {start.strip()} to {end.strip()} with weight {weight} added.")
                self.edge_entry.delete(0, tk.END)
            except ValueError:
                messagebox.showerror("Error", "Weight must be an integer.")
        else:
            messagebox.showerror("Error", "Please enter the edge in the format (start,end,weight).")

    def find_shortest_path(self):
        start = self.start_entry.get().strip()
        if start in self.graph.adjacency_list:
            distances = self.graph.dijkstra(start)
            result = "\n".join([f"Distance from {start} to {node}: {distances[node]}" for node in distances])
            self.result_text.delete(1.0, tk.END)
            self.result_text.insert(tk.END, result)
        else:
            messagebox.showerror("Error", f"Node {start} not found in the graph.")

    def plot_graph(self):
        G = nx.Graph()

        for node in self.graph.adjacency_list:
            G.add_node(node)

        for start, edges in self.graph.adjacency_list.items():
            for end, weight in edges.items():
                G.add_edge(start, end, weight=weight)

        pos = nx.spring_layout(G)
        labels = nx.get_edge_attributes(G, 'weight')

        nx.draw(G, pos, with_labels=True, node_color='lightblue', node_size=2000, font_size=10)
        nx.draw_networkx_edge_labels(G, pos, edge_labels=labels)
        plt.title("Graph Visualization")
        plt.show()

if __name__ == "__main__":
    root = tk.Tk()
    app = DijkstraApp(root)
    root.mainloop()
```

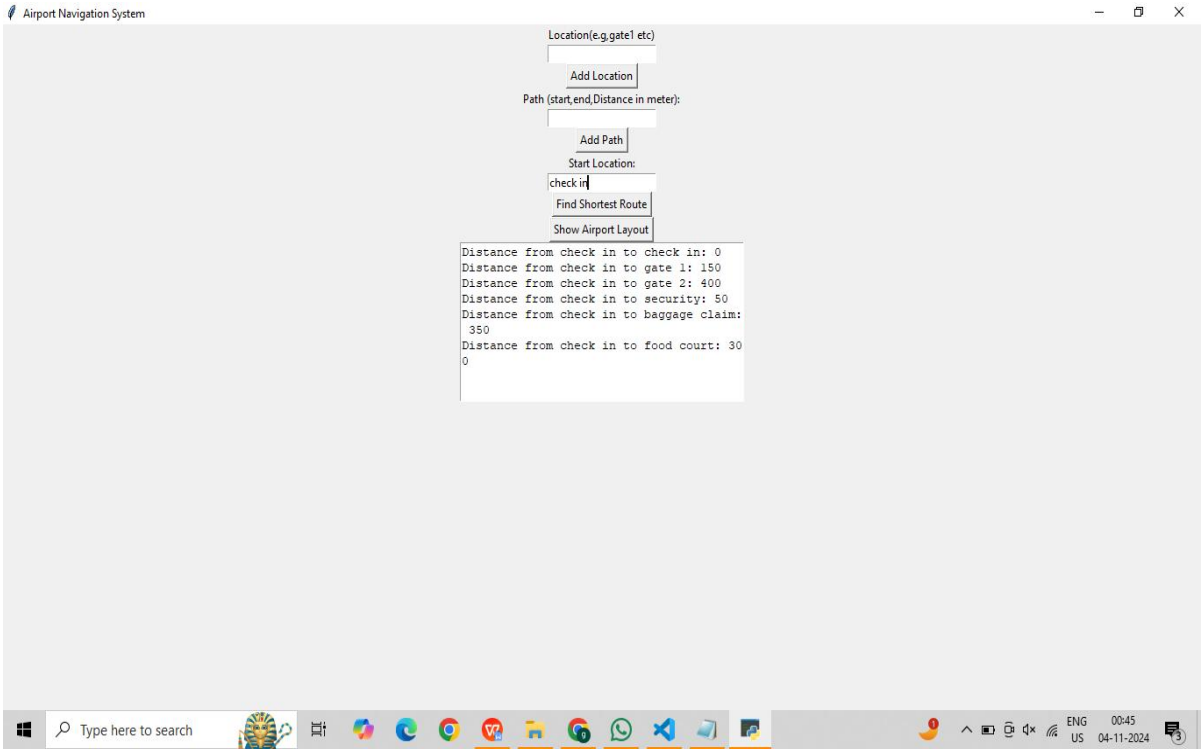
For Dijkastra Algorithm :

```
class Graph:
    def __init__(self):
        self.adjacency_list = {}

    def add_node(self, node):
        if node not in self.adjacency_list:
            self.adjacency_list[node] = {}

    def add_edge(self, start, end, weight):
        self.adjacency_list[start][end] = weight
        self.adjacency_list[end][start] = weight  # For undirected graph

    def dijkstra(self, start):
        distances = {node: float('infinity') for node in self.adjacency_list}
        distances[start] = 0
        visited = set()
        nodes = list(self.adjacency_list.keys())

        while nodes:
            current_node = min(
                nodes, key=lambda node: distances[node] if node not in visited else float('infinity')
            )
            nodes.remove(current_node)
            visited.add(current_node)

            for neighbor, weight in self.adjacency_list[current_node].items():
                if neighbor not in visited:
                    new_distance = distances[current_node] + weight
                    if new_distance < distances[neighbor]:
                        distances[neighbor] = new_distance

        return distances
```

Screenshots & Output






Airport Navigation System
Location(e.g,gate1 etc)
Add Location
Path (start,end,Distance in meter):
Add Path
Start Location:
check in
Find Shortest Route
Show Airport Layout
Distance from check in to check in: 0
Distance from check in to gate 1: 150
Distance from check in to gate 2: 400
Distance from check in to security: 50
Distance from check in to baggage claim: 350
Distance from check in to food court: 300
Type here to search
ENG US
00:45 04-11-2024

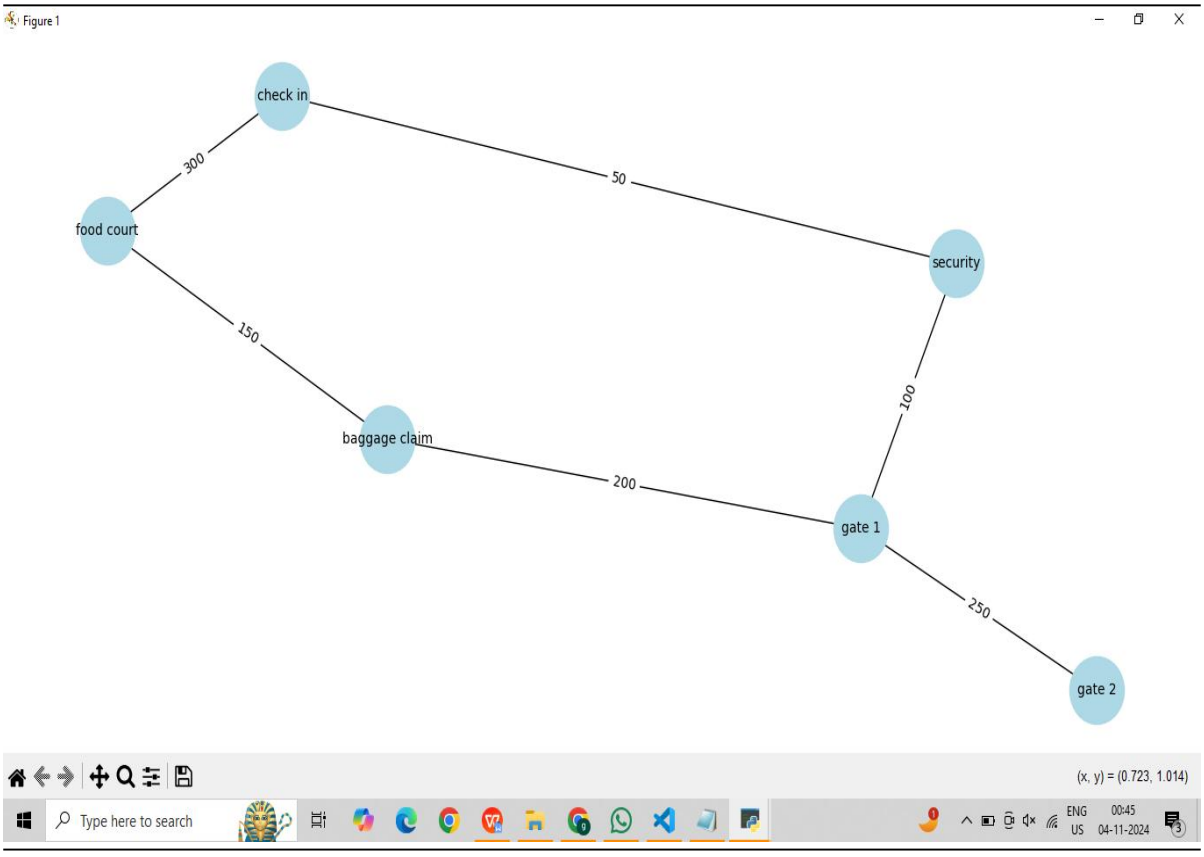


Figure 1
check in
food court
security
baggage claim
gate 1
gate 2
300
50
150
100
200
250
(x, y) = (0.723, 1.014)
Type here to search
ENG US
00:45 04-11-2024

Conclusion

The Airport Navigation System is a practical solution for helping travelers navigate airports efficiently. By combining Dijkstra's algorithm with an intuitive GUI, the system enhances user experience and minimizes confusion. Future enhancements could include features like real-time updates, integration with mobile devices, and additional routing options based on user preferences.

References

1. Dijkstra, E. W. (1956). "A note on two problems in connexion with graphs." Numerische Mathematik, 1(1), 269-271.
2. L. R. Lee, “An Introduction to Algorithms,” MIT Press, 2009.
3. B. A. Dave, "Algorithms and Data Structures for Graph Algorithms," Wiley, 2012.
4. Python Software Foundation. "Python Documentation." [Online]. Available: https://docs.python.org/3/
5. Matplotlib Development Team. "Matplotlib Documentation." [Online]. Available: https://matplotlib.org/stable/contents.html